

Label-efficient learning for High-Resolution Land Cover Classification using Large-Scale Self-Supervised Pre-Training and Transfer Learning

Dakota Hester

February 20, 2025

Abstract

Land cover data is a key component of environmental analysis and informs decision making in agriculture, urban planning, hydrology, and other applied domains. Most publicly available land cover products are produced at a coarse resolution due to the inherent difficulty in classifying high resolution data. Deep learning semantic segmentation models have been shown to be performant for such tasks but require large amounts of fully annotated tiles for training. This is problematic for land cover classification due to the expense and difficulty of annotating such data at scale. In our approach, we initially pre-train a ResNet-152 model on 377,913 unlabeled images using a denoising autoencoder to enable spatial feature extraction without the need for labeled data. This model is used as an encoder in a U-Net architecture where fully supervised pre-training is performed using 10,220 labeled images from another region. Finally, we fine-tune the U-Net with only 119 labeled images over the state of Mississippi. When applied to 1-meter imagery in Mississippi, our model achieves over 80

1 Introduction

Land cover data provides valuable information about the characteristics of the Earth’s surface for a wide variety of applications across many domains, including agriculture, urban planning, and environmental monitoring **CITE!**. Research into automated methods for land cover classification from remotely sensed data has been ongoing for several decades, with a majority of methods relying on supervised learning techniques to classify land cover using multispectral aerial or satellite imagery **CITE!**. Historically, these methods operate on a per-pixel basis, hand-engineered features are crafted from the raw data and used to train a classifier to predict the land cover class of each pixel in the image **CITE!**. However, these methods have shown to scale poorly with the increasing resolution of modern remote sensing data due to increased local variance and decreased inter-class separability **CITE!**.

Object-based Image Analysis (OBIA) techniques have been proposed as a potential workflow to address these issues, but their performance is highly dependent on the quality of objects or superpixels extracted from the imagery during the segmentation process **CITE!**.

Further, the segmentation process is often computationally expensive, and the parameters of the segmentation algorithm must be carefully tuned to achieve optimal results, resulting in poor generalization to areas outside of the training data **CITE!**. Deep learning methods have shown promise in addressing these issues by learning to extract hierarchical, spatially organized, translation invariant features from the raw data, then using these feature maps to predict the land cover class of each pixel in the image based on the presence of learned spectral and spatial patterns **CITE!**. Such methods have been shown to be effective at classifying land cover from high spatial resolution multispectral imagery at scale, addressing the limitations of traditional pixel-wise algorithms as well as OBIA-based methods **CITE!**. However, these methods often require large amounts of labeled training data to achieve optimal performance, which can be difficult to obtain due to the inherent cost of labeling such data at scale. Transfer learning has been shown to be effective at this solution by leveraging models pre-trained on large, diverse datasets to extract features

2 Related Works

3 Data

3.1 National Agriculture Imagery Program (NAIP)

3.2 Chesapeake Bay Land Use and Land Cover

3.3 In-house Labeled Data

Index	Class	Description
1	Open Water	Visible water bodies unobstructed by vegetation or structures. (e.g., rivers, ponds, lakes, creeks, reservoir).
2	Impervious Structures	
3	Impervious Surfaces	
4	Barren Land	
5	Forest/Woody Vegetation	
6	Herbaceous/Low Vegetation	
7	Cultivated Crops	
8	Unclassified	

Table 1: Land cover legend for our Mississippi dataset.

4 Methodology

4.1 Sampling

Ideally, land cover classification models should be trained and validated against samples that are representative of the study area. However, under the constraints of limited labeled data, it may be difficult to capture the full range of variation in the study area. Stratified sampling is a common approach to address this issue, where the study area is divided into strata based on some characteristic (such as land cover class or sub-region) and samples are drawn from each stratum. Unlike traditional land cover classification models, our semantic segmentation model necessitates the use of densely annotated tiles that exhibit a high degree of diversity with regards to spatial characteristics. Ideally, a single training tile should consist of a mix of land cover classes, and the arrangement of land cover classes should similarly be representative of the study area as a whole. To achieve this, we employ a stratified sampling strategy using the National Land Cover Database (NLCD) to guide the selection of training, validation, and testing tiles. This ensures that both the distribution of classes in the tiles is representative of the study area and the content of the tiles is heterogeneous and diverse in terms of spatial characteristics.

The NLCD provides land cover data at a 30m resolution for the entire United States, and the most recent edition (Annual NLCD) provides yearly land cover data between 1985 and 2023. We use 2023 NLCD data to ensure the land cover data is consistent with the NAIP imagery acquired during the same year. For our land cover product, we reclassify the level 1 land cover product to correspond with our choice of legend according to the mapping schema in Table ??.

A grid of 256 m x 256 m square tiles is overlaid over the study area, and the proportion of each land cover class is calculated for each cell. Applying principal component analysis (PCA) to the land cover relative frequency data, we identify that the first two principal components capture 94.24% of the data variance. Figure 1 visualizes the first two principal components of the land cover data at the tile level. Higher values of the first principal component correspond to a higher proportion of impervious land cover types in each tile, while higher values of the second principal component correspond to a higher proportion of forested and herbaceous land cover, with lower values correlating with regions that consist of tiles primarily or wholly composed of cultivated crops. These tiles are subsequently grouped into 250 clusters using K-means clustering operating on these two principal components. As shown in Figure 2, the clustering algorithm effectively divides the principal component space into regions that correspond to differing land cover compositions. From each cluster, we sample 2 candidate tiles for training, 1 for validation, and 1 for testing. This results in a total of 500 training tiles, 250 validation tiles, and 250 testing tiles.

Ensuring this strategy yields high-quality training data that is representative of the study area is crucial to the success of the model. However, we recognize that a rigid evaluation of such an approach to stratification is not possible due to the lack of ground truth data for the study area. One measure we take to ensure the stratification is effective is to inspect the distribution of land cover classes across the splits. Figure ?? shows that the distribution of land cover classes is consistent across the training, validation, and testing splits, which is ideal for ensuring that the model is exposed to a representative sample of the study

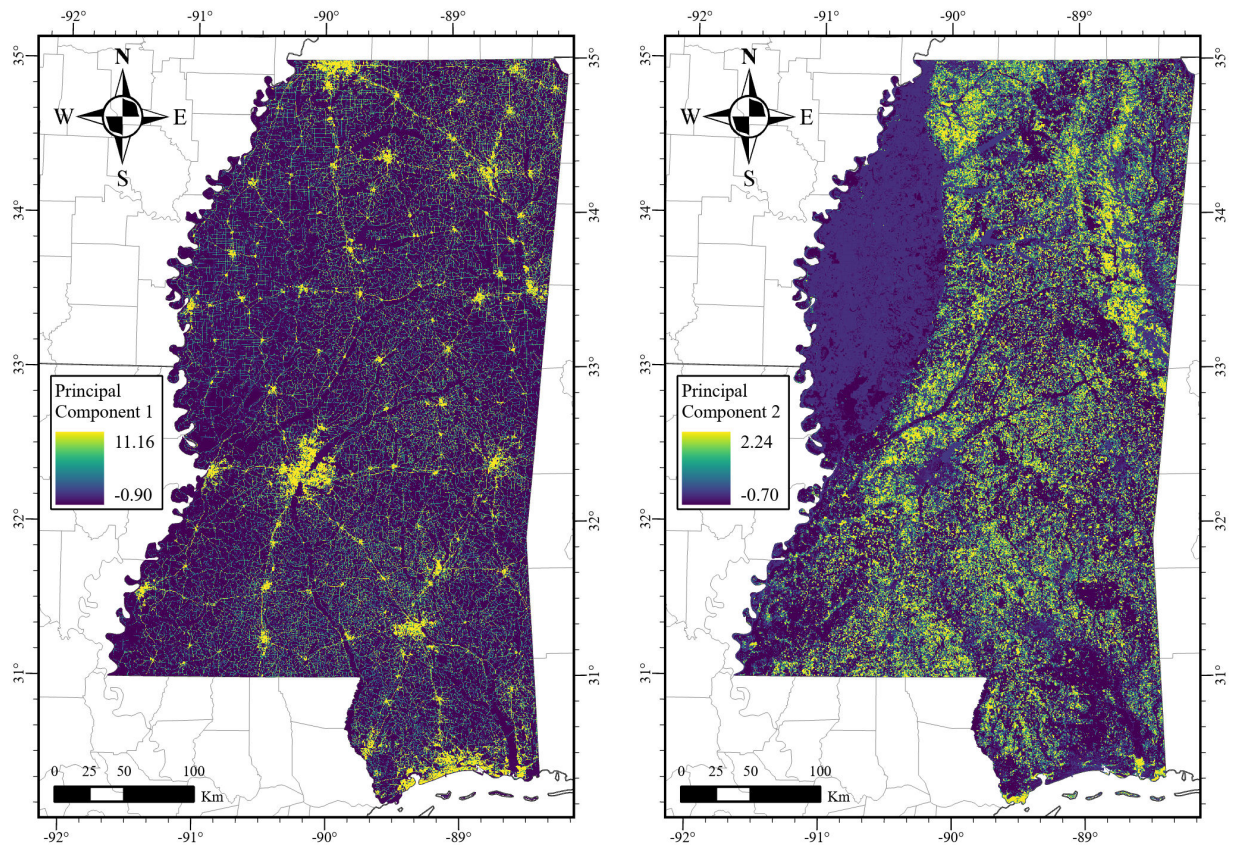


Figure 1: Principal components of NLCD relative frequencies per candidate tile.

area’s land cover diversity. One important note is that the distribution of land cover classes in the study area is highly imbalanced, with the majority of Mississippi’s land cover being classified as forest, and a low density of urban and developed areas. To tackle this issue, other strategies often employ synthetic oversampling of minority classes or strategic undersampling of majority classes **CITE**. Although our approach to stratifying the data does not explicitly enforce such policies, comparing the land cover class distribution in the sampled tiles to the distribution across the entire state (as shown in Figure ??) demonstrates that our sampling strategy achieves a similar outcome to these oversampling and undersampling strategies. Furthermore, our approach tends to avoid sampling tiles that are composed of a single land cover class. In the original NLCD data, 29.96% of the tiles are composed of a single land cover class, while only 1.90% of the sampled tiles are composed of a single land cover class. This is promising, as these homogeneous tiles are likely to be less informative for training the model **CITE**.

4.2 Model configuration

Our model is baed on the U-Net architecture, which is a popular choice for semantic segmentation tasks due to its relatively simple yet effective design [8] **OTHERS!!!**. First, in

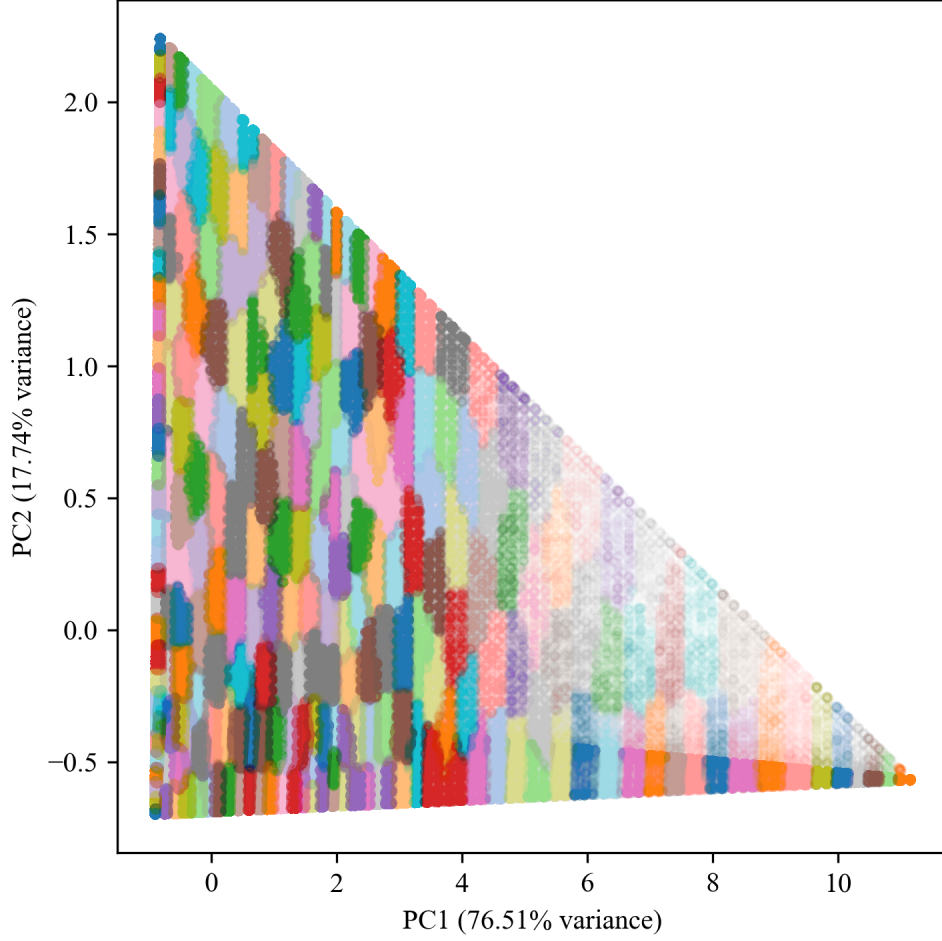


Figure 2: Clusters from which samples are draw in principal component space.

lieu of the original convolutional encoder network in the original implementation, we employ a much deeper ResNet-152 network as the encoder. Our reasoning for this choice is that the increased number of layers in the ResNet-152 network will facilitate more complex feature extraction, particularly during the self-supervised pre-training phase, where the large amount of unlabeled data will allow the encoder to learn to capture rich representations of spatial features in the input imagery. The number of channels accepted and produced by the decoder convolutions are subsequently adjusted to match the output of their corresponding encoder blocks. Second, we replace the transposed convolutional layers in the original U-Net decoder with bilinear upsampling to reduce the presence of checkerboard artifacts in the upsampled feature maps, which can inhibit the quality of the output segmentation maps [7, 9]. Finally, in each decoder block, we introduce a residual connection between the output of the first convolutional layer and the output of the second convolutional layer to facilitate the flow of information through the network and improve training stability [?].

4.3 Pre-training

Our pre-training approach consists of two phases: a self-supervised pre-training phase and a fully supervised pre-training phase.

Self-supervised pre-training In the first stage of pre-training, we train a denoting autoencoder using the unannotated NAIP imagery over the state of Mississippi. Our denoising autoencoder consists of a ResNet-152 encoder network and a simple decoder network that receives feature maps from each stage of the encoder network, resamples them to the original input resolution using bilinear interpolation, concatenates them, and passes them through a single 1×1 convolutional layer to produce an output with the same dimensions as the input. The ResNet-152 encoder network is initialized with ImageNet-1k pre-trained weights, while the decoder network is randomly initialized. A full decoder network (as is common in traditional autoencoders) is not used in this phase, as our primary goal is to force the encoder network to learn robust features present in the imagery that are necessary for reconstruction, rather than to necessarily learn to denoise the input images with a high degree of accuracy. Further, a U-Net architecture (like the one used in the subsequent fully supervised stages of training) are not used, as the presence of skip connections in the network can inhibit strong representation learning in the encoder network. The single-layer decoder network is designed to ensure that backpropagation primarily influences the parameters of the encoder network.

Two types of noise are added to the image: Gaussian noise and Poisson noise. The intensity of the Gaussian noise is controlled by the standard deviation parameter σ , which is randomly sampled from a uniform distribution $U(0, 2)$. Similarly, the intensity of the Poisson noise is controlled by the rate parameter λ , which is also randomly sampled from a uniform distribution $U(0, 1)$. Since random variables sampled from a Poisson distribution are strictly non-negative, Poisson noise is both added and subtracted from the input image (using the same λ value) to increase the impact of the noise on the input image. Model inputs are standardized to have zero mean and unit variance on a per-band basis using the mean and standard deviation of the pre-training data prior to adding noise. We opt for a relatively small batch size of 8 to reduce the memory requirements of the model during pre-training and to better approximate the stochasticity of the training data, improving the model’s generalization ability — an important consideration given our goal of transferring the encoder network to a downstream task [2, 5]. The loss function used during this self-supervised pre-training phase is the mean absolute error between the model’s raw output and the normalized input image. While the mean squared error is a common choice for autoencoder models, we opt for the mean absolute error as it tends to produce sharper images in the output [10, 6]. The AdamW optimizer is used to train the model, with an initial learning rate of $1e-6$ and a weight decay of $1e-2$ [4]. After each epoch of pre-training, the model is evaluated using the pre-training validation set to ensure that the model is fitting the data properly and to adjust the learning rate if necessary. If the validation loss does not decrease after an epoch, the learning rate is reduced by a factor of 0.1. While such a scheduler is aggressive, given the large volume of samples in the pretraining dataset and the sheer number of parameter updates that occur within a single epoch, we find that the model tends to optimize quickly and that there is little benefit to continuing to train at a learning rate where previous epochs have not shown improvement. Further, if the pretraining validation loss does not decrease for

5 consecutive epochs, the pre-training phase is terminated early. As such, while the model is trained for a maximum of 100 epochs, our model is capable of converging in 9 epochs **ADD FIGURE FOR PRE-TRAINING LOSS CURVE**. In order to estimate the quality of the reconstructions, we track the peak signal-to-noise ratio (PSNR) and structural similarity index (SSIM) between the input and output images during training [1].

FIGURE: DIFFERENT TYPES OF NOISE

Fully supervised pre-training Once the self-supervised pre-training phase is complete, the encoder network is used as the encoder in the U-Net architecture, and the decoder network is initialized with random weights. The model is trained to minimize the Focal loss, defined as:

$$\mathcal{L}_{\text{Focal}}(p_c) = \begin{cases} -\alpha_c (1 - p_c)^\gamma \log(p_c) & \text{if } y = c \\ -(1 - \alpha_c) p_c^\gamma \log(1 - p_c) & \text{otherwise} \end{cases} \quad (1)$$

where p_c is the predicted probability of class c , y is the true class label, α_c is the class weight for class c , and γ is a hyperparameter that controls the degree of class weighting. In order to address the class imbalance present in the data (**REFERENCE FIGURE HERE**, we employ a class weighting strategy that assigns class weights according to the formula:

$$\alpha_c = \frac{(1 - \rho_c)^\beta}{\sum_{c=1}^C (1 - \rho_c)^\beta} \quad (2)$$

where ρ_c is the proportion of class c in the training set, β is a hyperparameter that controls the degree of class weighting, and C is the total number of classes. For our experiments, we set $\gamma = 2.0$ in accordance with the authors' findings in the original Focal loss paper [3], and $\beta = 2.0$ such that minority classes are emphasized without majority classes being under-represented in the loss function. Like the previous pre-training stage, the model's parameters are updated using the AdamW optimizer, with an initial learning rate of Like the previous pre-training stage, the model is evaluated after every epoch using the validation set to adjust the learning rate and terminate the training early if necessary: if no improvement is observed in the validation loss for 3 consecutive epochs, the learning rate is reduced by a factor of 0.1, and if no improvement is observed for 15 consecutive epochs, the training is terminated early.

The model

4.4 Fine-tuning

4.5 Inference

Once the models have been trained and evaluated, they are

Predicted Label True Label	Open Water	Impervious Structures	Impervious Surfaces	Barren Land	Tree Canopy/ Woody Vegetation	Low Vegetation/ Herbaceous	Cultivated Crops	Unclassified	Producer's Accuracy
Open Water	7154	344	41	45	16	141	0	1066	81.23%
Impervious Structures	13	20776	2041	1335	115	4891	0	1708	67.28%
Impervious Surfaces	19	2590	124159	9058	1722	14407	13	3225	80.00%
Barren Land	298	1964	21482	130287	2798	100670	24151	4008	45.61%
Tree Canopy/ Woody Vegetation	924	244	1403	1064	2336744	94750	26166	101005	91.20%
Low Vegetation/ Herbaceous	469	2596	24339	57498	58379	851272	136447	29850	73.33%
Cultivated Crops	0	0	171	3079	6685	48908	148924	603	54.73%
Unclassified	208	940	3546	384	14128	8236	106	147915	62.64%
User Acc.	78.75%	70.54%	70.07%	64.26%	96.53%	75.78%	44.34%	51.11%	Overall Accuracy 82.12%

Table 2: Confusion matrix for the Mississippi dataset **REFERENCE ME!**.

4.6 Evaluation

5 Preliminary Results

6 Discussion

6.1 Early Conclusions

6.2 Potential impact on high-resolution remotely sensed imagery analysis

6.3 Future Methodology Improvements

6.3.1 Improved self-supervised pre-training phase

Stronger pre-text tasks

Higher-resolution feature maps

Deep supervision

Switch to AdamW optimizer

Pre-training using full-size images

6.3.2 Fine-tuning adjustments

More training and validation data

No oversampling

Simpler loss function (e.g., Focal Loss)

6.3.3 Inference changes

Longer stride

Exploiting rotation invariance using test-time augmentation

Post-processing

7 Conclusion

References

- [1] Alain Horé and Djemel Ziou. Image Quality Metrics: PSNR vs. SSIM. In *2010 20th International Conference on Pattern Recognition*, pages 2366–2369, August 2010.
- [2] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima, February 2017.
- [3] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollar. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.
- [4] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, January 2019.
- [5] Dominic Masters and Carlo Luschi. Revisiting Small Batch Training for Deep Neural Networks, April 2018.
- [6] Aamir Mustafa, Aliaksei Mikhailiuk, Dan Andrei Iliescu, Varun Babbar, and Rafal K. Mantiuk. Training a Task-Specific Image Reconstruction Loss. *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 21–30, January 2022.
- [7] Augustus Odena, Vincent Dumoulin, and Chris Olah. Deconvolution and Checkerboard Artifacts. *Distill*, 1(10):e3, October 2016.
- [8] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation, 2015.
- [9] Karshiev Sanjar, Olimov Bekhzod, Jaeil Kim, Jaesoo Kim, Anand Paul, and Jeonghong Kim. Improved U-Net: Fully Convolutional Network Model for Skin-Lesion Segmentation. *Applied Sciences*, 10(10):3658, January 2020.

- [10] Hang Zhao, Orazio Gallo, Iuri Frosio, and Jan Kautz. Loss Functions for Image Restoration With Neural Networks. *IEEE Transactions on Computational Imaging*, 3(1):47–57, March 2017.

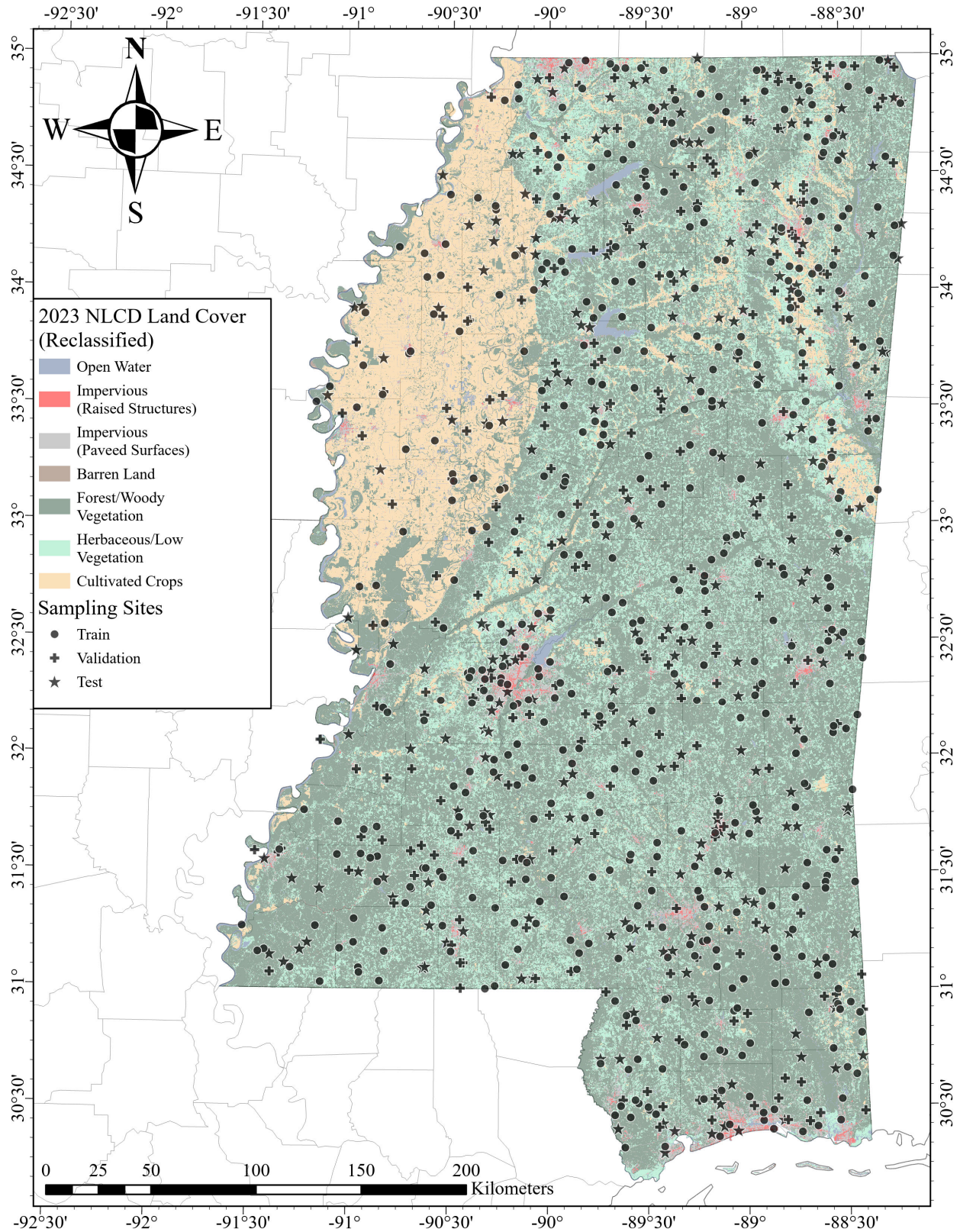


Figure 3: Sample locations for the Mississippi dataset **REFERENCE ME!!!**.

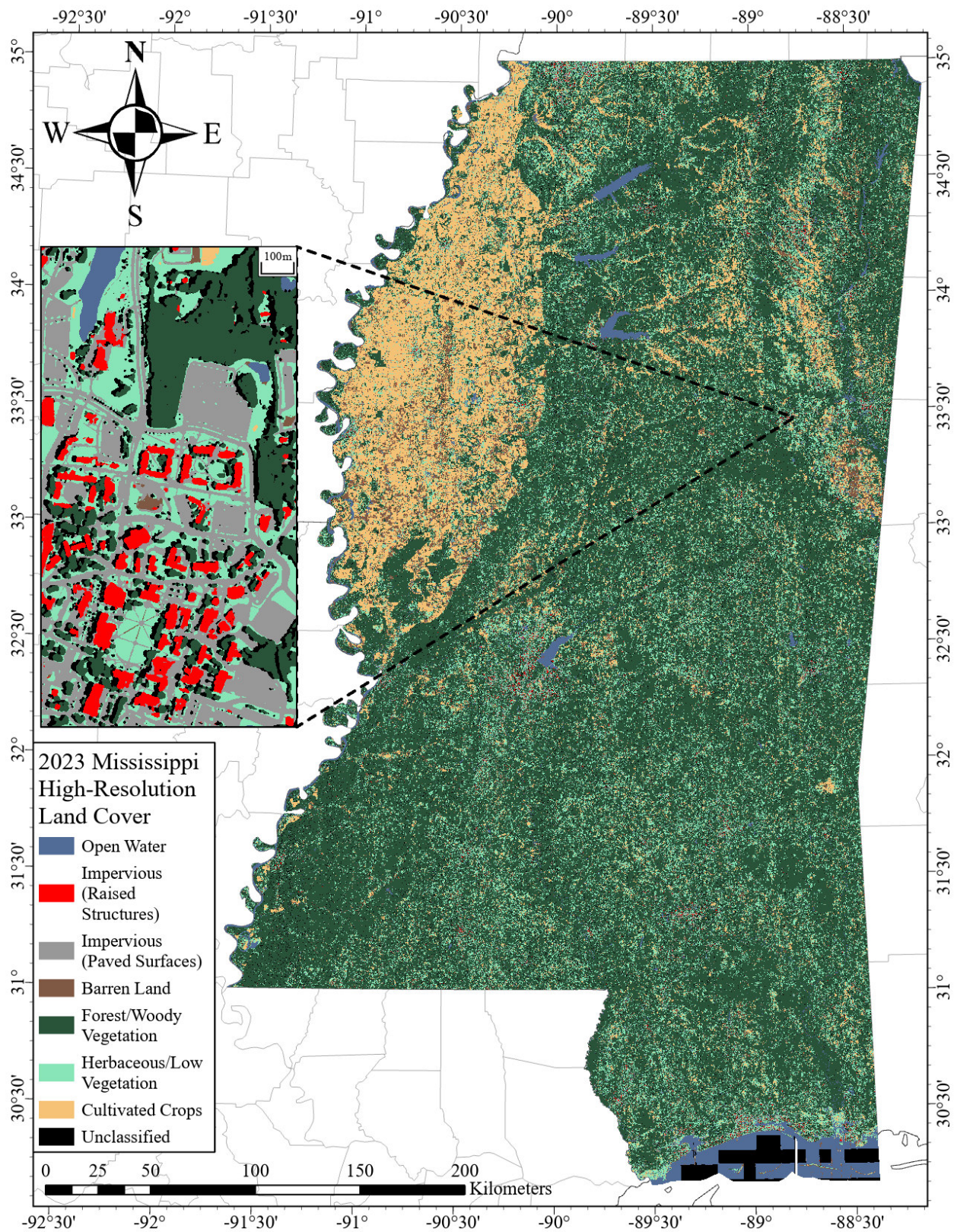


Figure 4: Preliminary high resolution land cover map of Mississippi **REFERENCE ME!!!**.